Professional Profile

Applied AI developer focused on building Retrieval-Augmented Generation (RAG) applications that answer questions from real documents and live databases. Experienced with LlamaIndex, vector embeddings, text-to-SQL, ReAct agents, and Streamlit interfaces. Built a complete banking assistant as a portfolio project.

Area	Tools and techniques
LLM applications	LlamaIndex, OpenAI GPT-4o-mini, prompt design, system prompts
Retrieval (RAG)	VectorStoreIndex, OpenAI text-embedding-3-small, top-k retrieval
Data access	NLSQLTableQueryEngine (text-to-SQL), SQLAlchemy, SQLite
Agents	ReActAgent with multiple QueryEngineTools and tool descriptions
Interface	Streamlit chat UI with cached resources
Language and tooling	Python, pip requirements, project structure, README docs

- Designed a hybrid system that routes each question to document search or to a SQL database.
- Grounded policy answers in source documents to reduce made-up fees and rules.
- Made database questions usable by non-technical staff through natural language.
- Packaged the work with setup scripts, sample data, a README, and a clear project layout.

Luxna Ramnauth

Project Portfolio - Gen AI Application Development

Screenshots are illustrative mockups of the interface.

Project Case Studies

The banking assistant is built from three modules. Each module solves one part of the problem, and an agent combines them.

Project 1: Banking Policy Q&A (RAG Module)

Problem. Bank policies are long documents. Staff and customers need exact answers about fees, loan limits, and card-security rules, and a general chatbot may invent rules that do not exist.

Solution.

- Split policy documents in `docs/` into chunks and convert them to embeddings.
- Stored the embeddings in a `VectorStoreIndex` and retrieve the top 3 chunks per question.
- Instructed the model to answer from the retrieved text only, so each answer matches the policy.

Skills demonstrated: RAG pipelines, vector embeddings, retrieval tuning, grounded prompting

Status: Implemented and wired into the agent as the *policy_search* tool.

Project 2: Customer Data Assistant (Text-to-SQL Module)

Problem. Account balances and transactions sit in a database. Non-technical users cannot write SQL, and a language model cannot see the data without a way to query it.

Solution.

- Built a SQLite database (`setup_db.py`) with customers, accounts, and 60 days of transactions.
- Used `NLSQLTableQueryEngine` so the model reads the table schema and writes SQL.
- Returned answers in plain language, and the generated SQL remains visible for checking.

Skills demonstrated: SQLAlchemy, schema-aware prompting, text-to-SQL, relational data modelling

Status: Database setup verified: 4 customers, 5 accounts, 300 transactions created.

Project 3: Agent Routing and Streamlit Interface

Problem. Two data sources need one entry point. A fixed if/else router would be hard to extend, and users need a simple chat interface.

Solution.

- Wrapped each engine as a `QueryEngineTool` with a clear description.
- Used a `ReActAgent` to reason about which tool to call and combine the results.
- Built a Streamlit chat page with cached resources so the index and database load once.
- Added safety rules to the system prompt against revealing OTPs or card numbers.

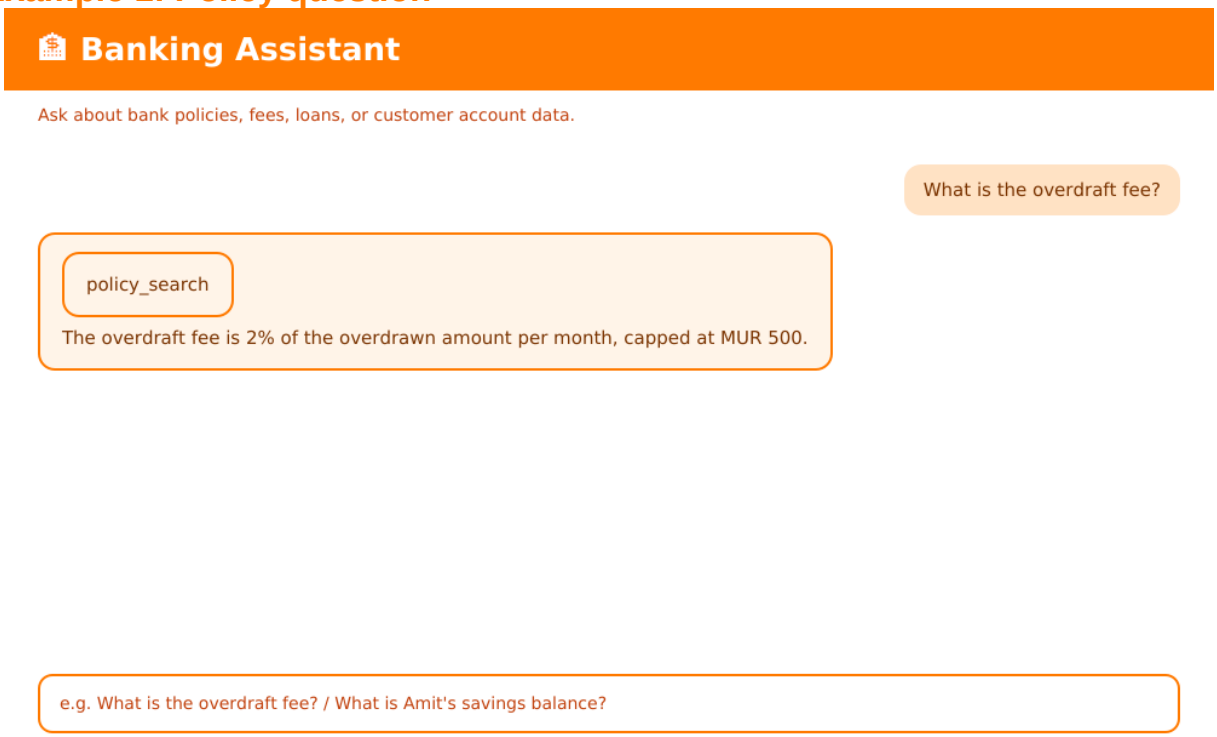
Skills demonstrated: ReAct agents, tool design, Streamlit, application safety rules

Status: Code compiles; full end-to-end run pending an OpenAI API key.

How the Application Works

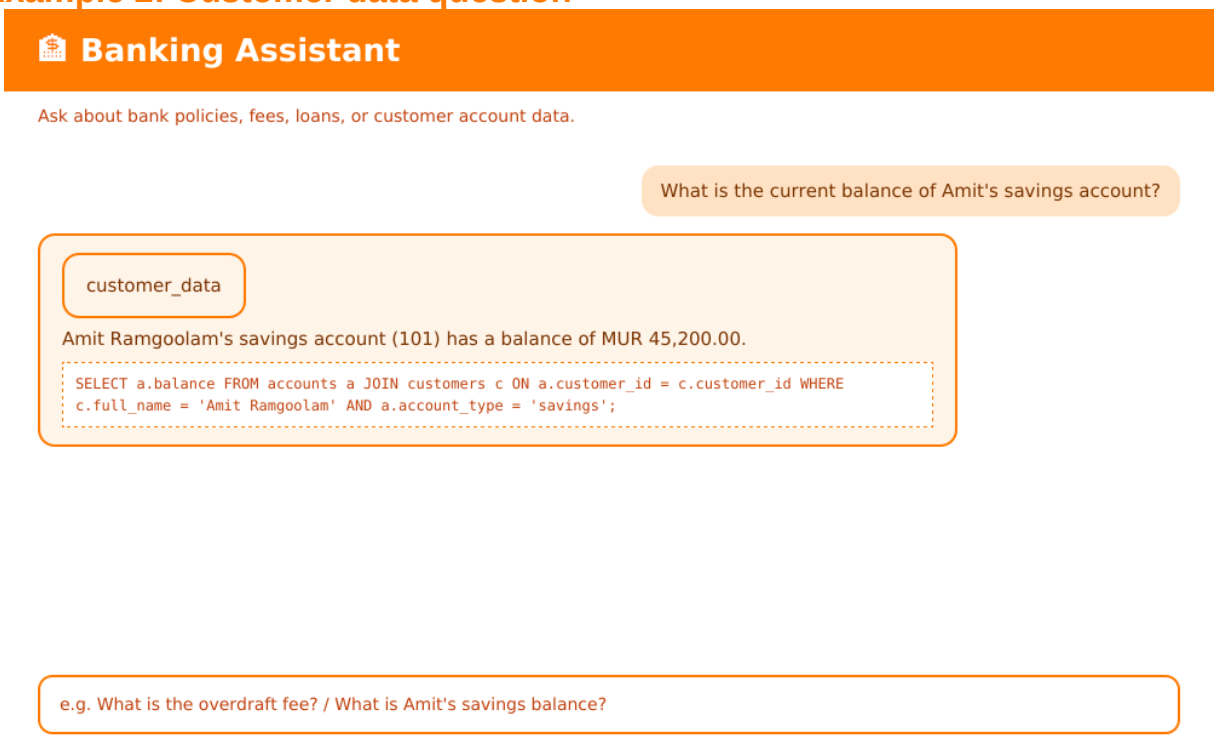
Each screenshot is an illustrative mockup of the interface, using the sample data.

Example 1: Policy question



The agent sends the overdraft-fee question to the policy search, which returns the 2% per-month fee capped at MUR 500.

Example 2: Customer data question



The agent sends the balance question to the customer data tool, which generates SQL and returns MUR 45,200.00 for Amit's savings account.

Next Steps

- Run the app with a live API key and capture real screenshots.
- Add PDF policy files and evaluate answer accuracy on a set of test questions.
- Restrict the SQL tool to read-only access and add PII masking before deploying.

Banking RAG Assistant

Luxna Ramnauth

Project Portfolio - Gen AI Application Development